

What are the drawbacks of docker?

Why Using Docker Alone in Production Is Not Recommended?

- ♦ **Single host only:**
Docker containers work on one dockerhost only. Managing many containers across multiple servers becomes very hard.
- ♦ **No auto-scaling:**
Docker doesn't scale containers up or down based on load. Everything must be done manually.
- ♦ **No auto-healing:**
If a container crashes or the machine goes down, Docker will not restart it or move it to another machine automatically. You have to do it yourself.
- ♦ **No load balancing or service discovery:**
Docker cannot distribute incoming traffic across containers on its own. You have to set this up manually.
- ♦ **Manual networking and storage:**
You must configure networking between containers and manage persistent storage yourself.
- ♦ **No deployment management:**
Docker alone can't handle rolling updates, rollbacks, or smooth version upgrades.

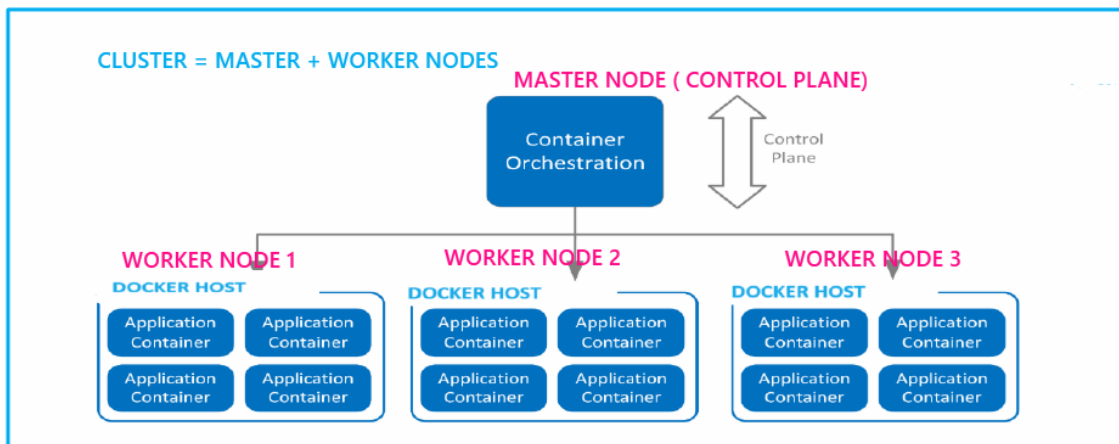
Conclusion:

Docker is fine for running one or two containers on a single machine. But for real production environments with many containers across many machines, Docker alone is not enough.. Kubernetes (or other orchestrators) add automation, scaling, self-healing, networking, and deployment management—making containerized apps truly production-ready.

Container orchestration:

Running docker containers across multiple machines is called container orchestration.

Instead of running all containers on one machine, you spread them across many machines. A control plane sits above all machines and manages everything.



Container orchestration tools:

Docker swarm / Kubernetes are the tools used for performing container orchestration.

Kubernetes is most widely used container orchestration tool in market.

If we are only running a single container or two containers together then we may not need an orchestrator.

How Container Orchestration Helps in overcoming docker container limitations?

Container orchestration tools like Kubernetes solve these problems:

1. Simplifies the container handling:

Orchestration tools take care running containers in different dockerhosts, which will help in managing containers.

2. Auto-Scaling of containers:

scaling means increasing / decreasing the number of containers.

When traffic goes up, Orchestration tools automatically adds more containers. When traffic drops, it removes them. All automatic.

3. High Availability:

if a container fails (or) exits, orchestration tools automatically restarts the failed container (or) it will replace failed containers with new containers thus ensuring availability.

4. **Networking:**

Orchestration tools can create & handle container networking, making it easier for containers to communicate with each other.

5. **Resource Efficiency:**

Orchestration tools help in controlling hardware resources used by containers.

6. **Load Balancing:**

rather than running all container in 1 docker host, we can distribute load (num of containers) to multiple docker host

In simple terms, container orchestration makes it easier to deploy, manage, and scale containers, ensuring that your applications run smoothly, reliably, and efficiently in a containerized environment.

Note:

1. **High availability (HA):**

means any application is expected to be up & running all time.

we should build an application environment, which should be highly available.

2. **Downtime:**

If an application goes down due to any issue, amount of time that application unavailable that time is downtime. we should build a system which should have zero downtime.

3. **What are the available containerization tools used ?**

Docker, containerD , Podman & CRI-O etc...

docker is most popular & widely used containerization tool

CRIO & containerD are very much similar to docker, used in Kubernetes

4. **What all container ORCHESTRATION technologies available ?**

Kubernetes, docker swarm, Mesos

Kubernetes is most popular & widely used container ORCHESTRATION

Assignment: Difference between Kubernetes & docker swarm

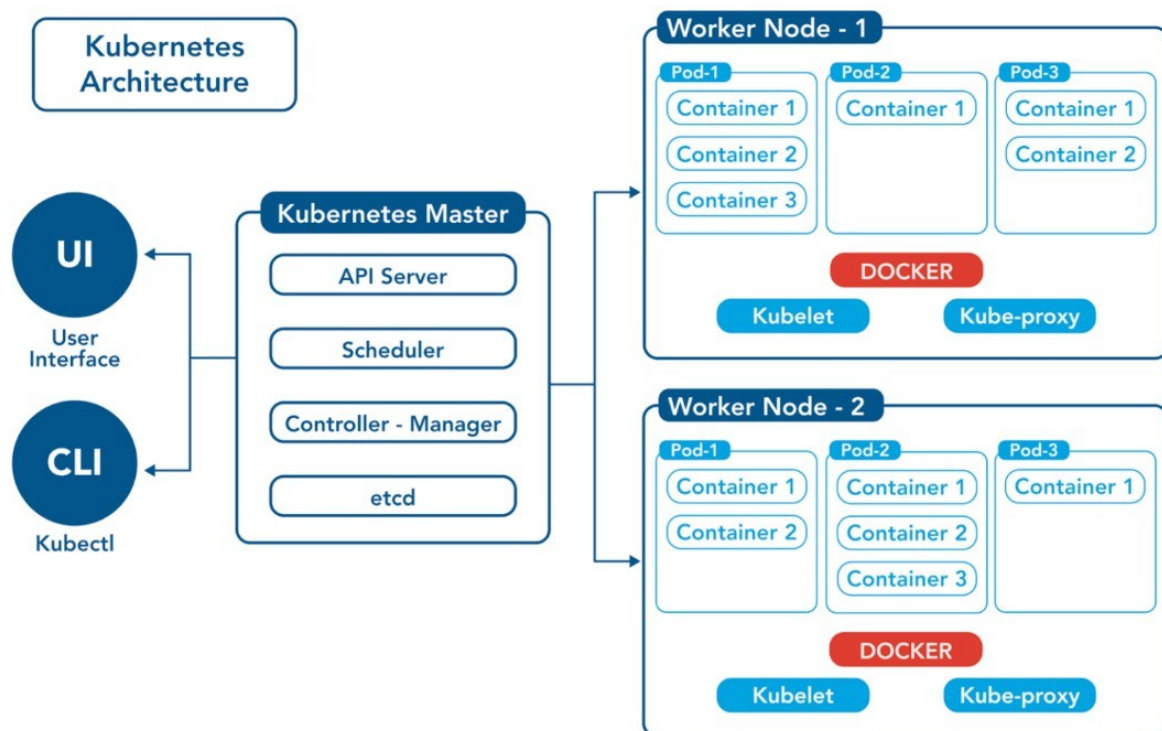
KUBERNETES:

- It is a container orchestration tool.
- Kubernetes is an open-source tool developed using GO programming language.
- Kubernetes is alias called as k8s
- k8s was developed by Google in 2014 it was made open source & donated to CNCF.
- k8s designed in master-slave type architecture.

Kubernetes creates cluster, where we will deploy our application in docker containers form. cluster is hosted by one node acting as the 'master' of the cluster, and other nodes(or dockerHost) as 'worker nodes (or slave nodes)' which do the actual 'containerization' using docker.

What is cluster in kubernetes?

→ cluster is a combination of k8s master node (control plane) + worker Nodes (docker hosts)



What is Master Node?

The master node is responsible for the management of entire Kubernetes cluster.

Master node is also called as **control plane**

it has 4 main components

1. kube-api server

2. kube scheduler
3. controller manager
4. etcd.

What are Worker Nodes?

Worker nodes are the nodes where the application (containers) actually running in Kubernetes cluster.

it has 3 main components

1. kubelet (agent)
2. kube-proxy
3. Container engine (docker)

Important Note on Naming conventions in kubernetes:

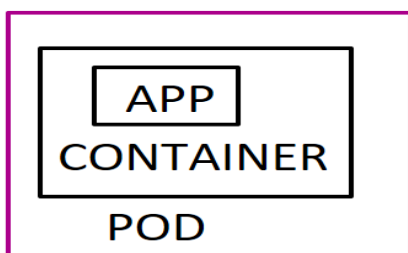
- i. servers==> vm == machine == nodes == slaves(workers) == instances ==> same
- ii. Kubernetes master is also called as control plane
- iii. Kubernetes nodes are also called as slave node or worker nodes
- iv. yaml files used in k8s are also called as definition files (or) manifest file.
- v. In git all commands starts with git eg: git pull , git push
In docker commands starts with eg: docker run , docker pull
In kubernetes its not kubernetes ==> its **kubect1** (Kube control)
eg: kubect1 get nodes, kubect1 get pods

Kubernetes is used to maximize the potential of containers – like scaling, healing, updating, and managing them automatically in production environments.

But **Kubernetes itself doesn't directly create containers.**

can Kubernetes directly create containers?

NO, Kubernetes does not create containers directly. It creates something called a **Pod** first, and the container runs inside that Pod.



Then how does Kubernetes run containers?

Kubernetes **creates something called a Pod**, not a container directly.

- A **Pod is the smallest deployable unit in Kubernetes**.
- Inside a Pod, **one or more containers will run**.
- So, containers are always **wrapped inside Pods** when managed by Kubernetes.